

Corrigés détaillés des annales Programmation Générique et Programmation Objet (2023 - 2025)

Ce document contient les corrigés détaillés des annales 2023, 2024 et 2025. Chaque exercice est expliqué avec : - les structures STL adaptées, - les classes nécessaires, - les fonctions demandées, - les algorithmes STL utilisés, - des exemples de code C++.

ANNALE 2023 — Jeu de Dames

Question 1 – Structures de données

Le damier peut être représenté avec :
`std::vector<std::vector<char>>`

Exemple :
`std::vector<std::vector<char>> damier(
 8,
 std::vector<char>(8, ' ')
);`

Chaque case contient :
- 'B' : pion blanc
- 'N' : pion noir
- ' ' : vide

Question 2 – Classe Dame

`class Dame {

private:

 std::vector<std::vector<char>> damier;
 char joueurCourant;

public:

 Dame();
 ~Dame();

 void afficher() const;

 bool deplacerPion(
 int xD,
 int yD,
 int xA,
 int yA
);
};`

Question 3 – Constructeur

Le constructeur initialise :
- le damier vide,
- les pions blancs,
- les pions noirs.

Exemple :
`Dame::Dame() {

 damier.resize(
 8,
 std::vector<char>(8, ' ')
);

 joueurCourant = 'B';
}`

Question 4 – Destructeur

Le destructeur peut être vide car STL libère

automatiquement la mémoire.

```
Dame::~Dame() {  
}
```

Question 5 – Déplacement

Le déplacement doit vérifier :

- les limites,
- la case vide,
- le déplacement diagonal.

Exemple :

```
if(damier[xA][yA] == ' ') {  
    damier[xA][yA] = pion;  
    damier[xD][yD] = ' ';  
}
```

ANNALE 2023 — Tables de données STL

Question 1 – Conteneur associatif

Le meilleur choix :

```
std::map<Cle, Valeur>
```

Pourquoi ?

- clés uniques,
- tri automatique,
- accès rapide.

Question 2 – Classe Cle

```
class Cle {  
  
private:  
  
    std::string matricule;  
  
public:  
  
    Cle(std::string m)  
        : matricule(m) {}  
  
    bool operator<(const Cle& c) const {  
        return matricule < c.matricule;  
    }  
};
```

Question 3 – Classe Valeur

```
class Valeur {  
  
private:  
  
    std::string nom;  
    std::string grade;  
    std::string telephone;  
  
public:  
  
    Valeur(  
        std::string n,  
        std::string g,  
        std::string t  
    )  
        : nom(n),  
          grade(g),  
          telephone(t) {}  
};
```

Question 4 – Couple clé-valeur

```
std::pair<Cle, Valeur> element(  
    Cle("0079"),  
    Valeur(  
        "HENRION",  
        "Lieutenant",  
        "2111"  
    )  
);
```

Question 5 – Construction table

```
std::map<Cle, Valeur> Table;
```

```
Table.insert(
    std::make_pair(
        Cle("0079"),
        Valeur(
            "HENRION",
            "Lieutenant",
            "2111"
        )
    )
);
```

Question 6 – Ajout de plusieurs éléments

```
Table.insert({
    Cle("0101"),
    Valeur("PIERRE", "Capitaine", "2211")
});
```

Question 7 – Affichage

```
for(auto it : Table) {
    std::cout << "Element"
               << std::endl;
}
```

ANNALE 2024 — Calcul de fréquences

Question 1 – Structures utilisées

```
std::vector<std::string>
std::map<std::string, int>
```

Question 2 – Classe Livre

```
class Livre {
private:
    std::vector<std::string> mots;
public:
    Livre(std::string fichier);
    std::vector<std::string> lexique();
    std::map<std::string, int> monogramme();
    void frequence1(std::string sortie);
    void frequence2(std::string sortie);
};
```

Question 3 – Constructeur

Lecture du fichier texte.

```
std::ifstream f(fichier);

std::string mot;

while(f >> mot) {
    mots.push_back(mot);
}
```

Question 4 – Fonction lexique

Utilisation d'un set pour supprimer les doublons.

```
std::set<std::string> ens;
```

Question 5 – Fonction monogramme

```
std::map<std::string, int> freq;
```

```

for(auto m : mots) {
    freq[m]++;
}

Question 6 – Sauvegarde lexicographique
-----
for(auto it : freq) {
    f << it.first
    << " "
    << it.second;
}

Question 7 – Tri par fréquence
-----
std::sort(
    vec.begin(),
    vec.end(),
    [](auto a, auto b) {
        return a.second > b.second;
    }
);

```

ANNALE 2024 — Emploi du temps

Question 1 – Structures

Une plage horaire contient :

- jour,
- début,
- fin,
- tâche.

Question 2 – Classe PlageHoraire

```

-----
class PlageHoraire {

private:

    std::string jour;
    int debut;
    int fin;
    std::string tache;

public:

    PlageHoraire(
        std::string j,
        int d,
        int f,
        std::string t
    );
};

```

Question 3 – Constructeur/destructeur

Le constructeur initialise les attributs.

Le destructeur n'est pas nécessaire car aucune mémoire dynamique n'est utilisée.

Question 4 – Code constructeur

```

-----
PlageHoraire::PlageHoraire(
    std::string j,
    int d,
    int f,
    std::string t
)
: jour(j),
  debut(d),
  fin(f),
  tache(t)
{
}

```

Question 5 – Setters

```

-----
void setJour(std::string j) {

```

```

    jour = j;
}

Question 6 – Programme de test
-----
PlageHoraire p(
    "lundi",
    10,
    12,
    "Tennis"
);

Question 7 – Classe EmploiTemps
-----
Le meilleur choix :
std::vector<PlageHoraire>

Question 8 – Implémentation
-----
class EmploiTemps {
private:
    std::vector<PlageHoraire> plages;
public:
    void ajouter(
        PlageHoraire p
    ) {
        plages.push_back(p);
    }
};

```

ANNALE 2025 — Bibliothèque numérique

```

Question 1 – Structures STL
-----
Le meilleur choix :
std::vector<Livre>

Pourquoi ?
- parcours simple,
- tri facile,
- compatible STL.

Question 2 – Classe Livre
-----
class Livre {
private:
    std::string titre;
    std::string auteur;
    int annee;
    int identifiant;
public:
    Livre(
        std::string t,
        std::string a,
        int an,
        int id
    );
    bool operator<(const Livre& l) const;
};

Question 3 – Implémentation
-----
bool Livre::operator<(
    const Livre& l
) const {
    return annee < l.annee;
}

Question 4 – Classe Bibliotheque
-----

```

```

Fonctions demandées :
- ajouterLivre
- supprimerLivre
- rechercherParAuteur
- afficherParAnnee
- compterLivresApres

Exemple suppression :
-----
livres.erase(
    remove_if(
        livres.begin(),
        livres.end(),
        [id](Livre l) {
            return l.getId()==id;
        }
    ),
    livres.end()
);

Question 5 – Itérateurs STL
-----
Les itérateurs permettent :
- parcourir les conteneurs,
- utiliser sort/count_if/remove_if.

Question 6 – main()
-----
Bibliotheque b;

b.ajouterLivre(
    Livre(
        "Livre1",
        "Hugo",
        1990,
        1
    )
);

Question 7 – Sauvegarde fichier
-----
std::ofstream f(
    "bibliotheque.txt"
);

```

ANNALE 2025 — Urgences écologiques

```

Question 1 – Classe
-----
class UrgenceEcologique {
public:
    std::string lieu;
    int niveau;
    int date;

    bool operator<(
        const UrgenceEcologique& u
    ) const {
        if(niveau == u.niveau)
            return date > u.date;

        return niveau < u.niveau;
    }
};

Question 2 – priority_queue
-----
std::priority_queue<
    UrgenceEcologique
> file;

Question 3 – Ajouter urgence
-----
void ajouterUrgence(
    std::priority_queue<
        UrgenceEcologique
    >& file,

```

```
    const UrgenceEcologique& u
) {
    file.push(u);
}
```

Question 4 – Traitement

```
-----
while(!file.empty()) {
    auto u = file.top();
    std::cout << u.lieu;
    file.pop();
}
```

Question 5 – Jeu de test

```
-----
ajouterUrgence(
    file,
    {"Amazonie", 95, 20250501}
);

ajouterUrgence(
    file,
    {"Australie", 100, 20250502}
);
```